

Assignment: Compiler Front-End Components, Built and Explained

Course: Programming Languages and Structures (CSC 461)

Department of Computer Science & Engineering, IUBAT

1. Introduction

Before a computer can run the code a programmer writes, that code has to pass through a translator called a compiler. A compiler does not do its job in a single pass — it works through a sequence of phases, and the earliest group of phases is what people call the front-end. The front-end is responsible for reading the raw source text, splitting it into meaningful pieces, confirming those pieces appear in a legal order, and finally checking that the program actually means something sensible.

This report walks through four front-end components that were implemented for this assignment. Task 1 splits the source program into lexemes. Task 2 sorts those lexemes into categories such as keywords, identifiers and operators. Task 3 assigns an id number to every variable and builds a lookup (symbol) table. Task 4 implements a parser based on the supplied grammar, prints the resulting parse tree, and performs type checking using an attribute grammar. Python was chosen for all four tasks because it keeps the implementation compact and easy to follow.

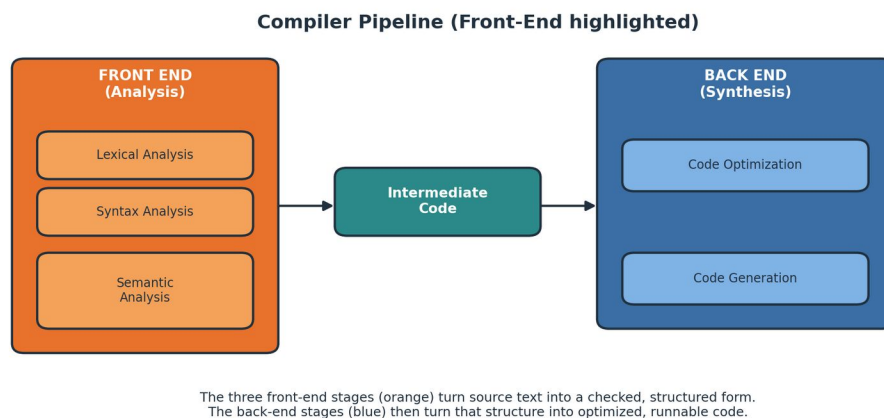


Figure 1: The compiler pipeline redrawn as a two-stage flow. The three orange boxes are the front-end stages this assignment implements; the blue boxes are the back-end stages that come afterward.

Tasks 1 through 3 were all run against the identical sample program supplied with the assignment brief, reproduced below exactly as given:

```
int func(int xyz)
{
    xyz=xyz+1;
    return xyz;
}

int main()
{
    int a, b=5;
    for(int i=0;i<=10;i++)
    {
```

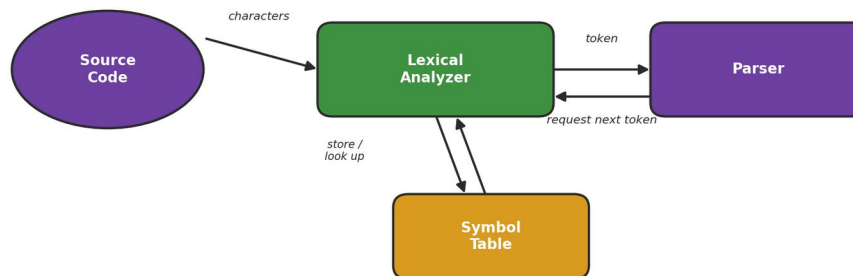
```
b=b+func(i);  
}  
return 0;  
}
```

2. Task 1 — Splitting the Source into Lexemes

What counts as a lexeme

A lexeme is the smallest chunk of text that still carries meaning on its own. A human glancing at `xyz=xyz+1;` sees a single line, but a compiler must recognize six distinct units inside it: `xyz`, `=`, `xyz`, `+`, `1`, and `;`. This task's job is exactly that — slice the source into those units and print them in the order they were found.

Where the Lexical Analyzer Sits



The analyzer consumes raw characters, groups them into tokens for the parser, and consults the symbol table whenever an identifier needs to be recorded or checked.

Figure 2: The lexical analyzer's place in the pipeline, redrawn with the source feeding characters in, tokens flowing out to the parser, and the symbol table consulted on the side.

Approach taken

No external tokenizing library was used. The source string is scanned one character at a time with an index pointer `i`, and the next action is chosen based on what that character looks like:

- Whitespace (space, tab, newline) is simply skipped — it separates lexemes but is never a lexeme in its own right.
- A letter or underscore starts a run of letters/digits, producing a full word — either a reserved keyword such as `int`, or an identifier such as `xyz`.
- A digit starts a run of digits (and possibly a decimal point), so that a value such as `10` or `2.6` is captured as a single token rather than being split apart.
- Before single-character symbols are tested, the next two characters are checked together against a table of two-character operators (`<=`, `++`, `==` and similar). Checking two-character combinations first is essential — testing single characters first would break `<=` into `<` followed by `=`.
- Anything left over that matches a known punctuation symbol, such as `( ) { }` or `;`, is emitted as its own one-character lexeme.

Always preferring the longest possible match before falling back to a shorter one is a standard scanning strategy known as the longest-match rule.

Implementation — task1_lexeme_separation.py

```
two_char_ops = ["==", "!=", "<=", ">=", "++", "--", "&&", "||", "+=", "-="]
one_char_ops = "+-*/%=<>!(){}[];,"
```

```
def separate_lexemes(code):
    lexemes = []
    i = 0
    n = len(code)
    while i < n:
        ch = code[i]
        # 1. skip blank space, tab and new line
        if ch in "\t\n\r":
            i = i + 1
            continue
        # 2. word : keyword, function name or variable name
        if ch.isalpha() or ch == "_":
            word = ""
            while i < n and (code[i].isalnum() or code[i] == "_"):
                word = word + code[i]
                i = i + 1
            lexemes.append(word)
            continue
        # 3. number (integer or real number)
        if ch.isdigit():
            num = ""
            while i < n and (code[i].isdigit() or code[i] == "."):
                num = num + code[i]
                i = i + 1
            lexemes.append(num)
            continue
        # 4. two character operator
        if code[i:i + 2] in two_char_ops:
            lexemes.append(code[i:i + 2])
            i = i + 2
            continue
        # 5. single character operator or punctuation
        if ch in one_char_ops:
            lexemes.append(ch)
            i = i + 1
            continue
        i = i + 1
    return lexemes
```

Result

The scanner identified 58 lexemes in total. To save space they are shown below across three columns; the actual program prints one lexeme per line, exactly as the assignment specifies.

Total lexemes found : 58

----- Lexemes -----

```
int    )    i
func   {    ++
(      int  )
int    a    {
xyz    ,    b
)      b    =
{      =    b
xyz    5    +
```

```

= ; func
xyz for (
+ ( i
1 int )
; i ;
return = }
xyz 0 return
; ; 0
} i ;
int <= }
main 10
( ;

```

This output was checked line by line against the sample output provided in the assignment brief, and the two match.

3. Task 2 — Sorting Lexemes into Categories

Once the lexemes exist, the compiler still has no notion of what any of them represent. This task assigns every lexeme to one of five buckets: Keywords, Functions, Variables, Operators and Others. Two fixed reference lists — one of keywords, one of operators — drive the classification, following the rules below:

Group	Rule applied	Examples produced
Operators	The lexeme appears in the operator list.	+, ++, <=, =
Keywords	The lexeme is a reserved word of the language.	for, int, return
Functions	The lexeme is a name directly followed by an opening parenthesis '('.	func, main
Variables	The lexeme is a name not followed by '('.	a, b, i, xyz
Others	Anything else — numbers, brackets, comma, semicolon.	() , ; { } 0 1 5 10

Telling a function apart from a variable relies on peeking one lexeme ahead. In func(i), the name func is immediately trailed by (, so it is classified as a function; in b=b+1, the name b is followed by =, so it is a variable instead. The identifier main is handled as a special case — it reads like a keyword in many textbooks, but functionally it is simply a function name.

Duplicate names also needed to be collapsed — xyz shows up four separate times in the source, yet it should be listed only once per group. sorted(set(items)) takes care of both removing repeats and ordering the group alphabetically.

Implementation — task2_lexeme_grouping.py

```

def group_lexemes(lexemes):
    keywords = []; functions = []; variables = []
    operators = []; others = []
    for pos in range(len(lexemes)):
        lex = lexemes[pos]
        if lex in OPERATORS:
            operators.append(lex)
        elif lex in KEYWORDS:
            if lex == "main": # main is really a function

```

```

        functions.append(lex)
    else:
        keywords.append(lex)
    elif lex.isidentifier():
        # if the next lexeme is "(" then this name is a function
        if pos + 1 < len(lexemes) and lexemes[pos + 1] == "(":
            functions.append(lex)
        else:
            variables.append(lex)
    else:
        others.append(lex)
    return keywords, functions, variables, operators, others

def clean(items):
    return sorted(set(items)) # remove duplicates and sort

```

Result

```

Functions: func, main
Operators: +, ++, <=, =
Variables: a, b, i, xyz
Keywords: for, int, return
Others: (, ), ,, 0, 1, 10, 5, :, {, }

```

This lines up exactly with the sample output that came with the assignment.

4. Task 3 — Symbol IDs and the Lookup Table

Production compilers do not pass variable names around forever. Each distinct name is registered once inside a symbol table (also called a lookup table), and everywhere else the compiler refers to it only by a compact id number. This is both faster and keeps every fact about a variable — its type, in this case — in a single place. This task assigns those id numbers, substitutes them for the raw names, and prints the completed lookup table.

Determining each variable's type

An id number alone is incomplete without a type to go with it. The lexeme list is scanned while tracking a running variable `current_type`: whenever a type keyword such as `int` or `float` appears, it is remembered, and any identifier seen afterward inherits that type. Reaching a `;` or a `)` signals the end of the declaration, so `current_type` is reset to `None`.

This mechanism also correctly handles a line like `int a, b=5;` — the word `int` only appears once, but because `current_type` isn't cleared until the semicolon, both `a` and `b` end up correctly typed as `int`.

Implementation — `task3_lookup_table.py`

```

def find_types(lexemes):
    types = {}
    current_type = None
    for pos in range(len(lexemes)):
        lex = lexemes[pos]
        if lex in TYPE_WORDS:
            current_type = lex
            continue
        if lex.isidentifier() and lex not in KEYWORDS:
            next_lex = lexemes[pos + 1] if pos + 1 < len(lexemes) else ""
            if next_lex == "(":
                continue # it is a function call

```

```

    if current_type is not None and lex not in types:
        types[lex] = current_type
    if lex == ";" or lex == " ": # declaration finished
        current_type = None
    return types

def build_lookup_table(variables, types):
    table = {}
    number = 1
    for name in variables:
        table[name] = (number, types.get(name, "int"))
        number = number + 1
    return table

```

Result

Functions: func, main
 Variables: <1 id> <2 id> <3 id> <4 id>
 Operators: +, ++, <=, =
 Keywords: for, int, return
 Others: (,), ,, 0, 1, 10, 5, :, {, }

Lookup table:
 id variable type
 1 a int
 2 b int
 3 i int
 4 xyz int

For readability, the same lookup table is repeated below as a formatted table, with a short note on where each variable comes from:

id	Variable	Type	Declared where
1	a	int	inside main(), on the line int a, b=5;
2	b	int	inside main(), on the line int a, b=5;
3	i	int	inside the for loop, for(int i=0; ...)
4	xyz	int	parameter of the function func()

In the grouped output, the four variable names are replaced by <1 id>, <2 id>, <3 id> and <4 id>, matching the sample output. The lookup table remains the single source that maps each id back to its real name and type — which is precisely why compilers maintain such a table in the first place.

5. Task 4 — Parsing, the Parse Tree, and Semantic Checking

Grammar followed

The parser implements the BNF grammar supplied with the assignment: a program consists of a function; a function has a type, a name, parameters, and a body of statements; and a statement may be a declaration, an assignment, or a return. Expressions are structured across three levels — Expression, Term and Factor — which is exactly what gives multiplication and division higher precedence than addition and subtraction.

Why recursive descent

A recursive-descent (top-down) parser was chosen. Each grammar rule maps to one function of almost the same name — `parse_expression()` implements `<Expression>`, `parse_term()` implements `<Term>`, and so on. When one rule references another, the corresponding function simply calls the other. The resulting code mirrors the grammar closely, which makes it straightforward to verify and to debug.

One wrinkle: the rule `<Expression> ::= <Expression> "+" <Term>` is left-recursive — it references itself as its very first step. Implemented literally inside a recursive-descent parser, `parse_expression()` would call itself endlessly without ever consuming a token, crashing with infinite recursion. This was avoided by rewriting the rule as a loop: read one term first, then keep reading an operator and another term for as long as the next lexeme is `+` or `-`. The language accepted is unchanged, and the operators remain left-associative.

Constructing the parse tree

Every parsing function builds one `Node` object and attaches whatever nodes its sub-calls return as children. By the time parsing completes, the nodes already form a tree, which a small recursive routine then prints with indentation to show its shape.

```
class Node:
    def __init__(self, name, value=None):
        self.name = name      # name of the grammar symbol
        self.value = value    # only for leaf nodes (id, number ...)
        self.children = []    # child nodes
        self.type = None      # synthesized attribute : data type

    def add(self, child):
        self.children.append(child)
        return child

    def show(self, level=0):
        pad = " " * level
        if self.value is None:
            print(pad + "<" + self.name + ">")
        else:
            print(pad + "<" + self.name + "> : " + str(self.value))
        for c in self.children:
            c.show(level + 1)
```

Recursive-descent parsing functions

```
# <Expression> ::= <Expression> "+" <Term> | <Expression> "-" <Term> | <Term>
# left recursion is removed by using a loop
def parse_expression(self):
    node = Node("Expression")
```

```

left = self.parse_term()
node.add(left)
result_type = left.type
while self.look() in ["+", "-"]:
    op = self.take()
    node.add(Node("operator", op))
    right = self.parse_term()
    node.add(right)
    result_type = self.result_type(result_type, right.type, op)
node.type = result_type # attribute passed up the tree
return node

# <Factor> ::= id | number | "(" <Expression> ")"
def parse_factor(self):
    node = Node("Factor")
    tok = self.look()
    if tok == "(":
        self.take()
        inner = self.parse_expression()
        node.add(inner)
        self.match("(")
        node.type = inner.type
    elif self.is_number(tok):
        self.take()
        leaf = Node("number", tok)
        leaf.type = "float" if "." in tok else "int"
        node.add(leaf)
        node.type = leaf.type
    elif tok.isidentifier():
        self.take()
        leaf = Node("id", tok)
        if tok in self.symbol_table:
            leaf.type = self.symbol_table[tok]
        else:
            leaf.type = "unknown"
            self.errors.append("Semantic Error: variable '" + tok +
                               "' is used without declaration.")
        node.add(leaf)
        node.type = leaf.type
    return node

```

Type checking through an attribute grammar

Syntax checking only confirms that a program is shaped correctly — it cannot say whether the program is meaningful. $y = x + 1$; is perfectly well-formed, for instance, but if y is declared `int` while x is `float`, the assignment cannot happen safely. Spotting that kind of problem is what semantic analysis is for.

Semantic checking here is implemented as an attribute grammar. Every parse-tree node carries a type attribute. Because this attribute is computed at the leaves and then combined upward toward the root, it is called a synthesized attribute. A literal like 5 is typed `int`, while 2.6 is typed `float` because it contains a decimal point; a variable's type is pulled straight from the symbol table. Each parent node then derives its own type from its children according to the following rules:

Grammar rule	Semantic rule applied
Factor $\rightarrow$ number	type = float if the literal has a decimal point, else int
Factor $\rightarrow$ id	type = lookup(id) from the symbol table
Factor $\rightarrow$ (Expression)	type = Expression.type

Grammar rule	Semantic rule applied
Term $\rightarrow$ Term * Factor	type = float if either side is float, else int
Expression $\rightarrow$ Expression + Term	type = float if either side is float, else int
Assignment $\rightarrow$ id = Expression	check whether Expression.type can be stored in lookup(id); report an error if not

Two helper routines carry out the actual checks: `result_type()` works out the combined type when two values meet at an operator, and `check_assignment()` decides whether the computed value may legally be stored in the target variable. Consistent with the sample outputs supplied, storing an int value into a float variable is flagged as an error, and combining two unrelated types (such as string and int) is reported as a type mismatch.

```
def result_type(self, left_type, right_type, op):
    numbers = ["int", "float"]
    if left_type in numbers and right_type in numbers:
        if left_type == "float" or right_type == "float":
            return "float"
        return "int"
    if left_type == right_type:
        return left_type
    self.errors.append("Type Compatibility Error: Type mismatched.")
    return "unknown"

def check_assignment(self, name, left_type, right_type):
    if right_type == "unknown" or left_type == "unknown":
        return
    numbers = ["int", "float"]
    if left_type not in numbers or right_type not in numbers:
        if left_type != right_type:
            self.errors.append("Type Compatibility Error: Type mismatched.")
        return
    if left_type == "float" and right_type == "int":
        self.errors.append(
            "Type Compatibility Error: Can't assign float to int.")
```

A further design choice worth noting: the parser never stops at the first problem it finds. Every issue is appended to a running list, `self.errors`, and parsing continues to the end — so a user gets to see every defect in the program from a single run instead of fixing mistakes one at a time.

Running the three supplied test cases

All three test programs given in the assignment brief were run through the parser, and each produced the expected verdict.

Test Case 1 — a well-typed program

```
int x;
float y;
x = 5;
y = 2.6;
x = y+1;
```

Semantic Analysis Result: Passed

Here $y+1$ evaluates to float, and storing a float value into x is permitted, so the program clears the semantic check.

Test Case 2 — narrowing a float into an int

```
int x;  
float y;  
x = 5;  
y = 2.6;  
y = x+1;
```

Semantic Analysis Result: Type Compatibility Error: Can't assign float to int.

Test Case 3 — unrelated types

```
int x;  
string t;  
x = 8;  
t = x;
```

Semantic Analysis Result: Type Compatibility Error: Type mismatched.

Here t is a string and x is an int, and the two share nothing in common, so the checker correctly flags a type mismatch — matching the expected output for this case.

Parse tree produced for Test Case 1

The tree shown below is the literal output the program printed. Each indent level marks one level further down the tree. Notice how the final statement, $x = y + 1$;, becomes an Assignment node whose Expression child holds a Term for y , the $+$ operator, and a Term for 1 .

```
<Program>  
  <Statement>  
    <Declaration>  
      <Type> : int  
      <id> : x  
    <Statement>  
      <Declaration>  
        <Type> : float  
        <id> : y  
      <Statement>  
        <Assignment>  
          <id> : x  
          <Expression>  
            <Term>  
              <Factor>  
                <number> : 5  
          <Statement>  
            <Assignment>  
              <id> : y  
              <Expression>  
                <Term>  
                  <Factor>  
                    <number> : 2.6  
          <Statement>  
            <Assignment>  
              <id> : x  
              <Expression>  
                <Term>  
                  <Factor>  
                    <id> : y
```

```

<operator> : +
<Term>
  <Factor>
    <number> : 1

```

Symbol table: {'x': 'int', 'y': 'float'}

Semantic Analysis Result:

Passed

How the type attribute travels up the tree

The table below traces, step by step, exactly how the attribute grammar resolves the type of the final line, $x = y + 1$; — this is the mechanism actually at work behind the scenes.

Step	Tree node	Rule applied	Type produced
1	Factor $\rightarrow$ id (y)	type comes from the symbol table	float
2	Term $\rightarrow$ Factor	type is copied up from the child	float
3	Factor $\rightarrow$ number (1)	no decimal point present	int
4	Term $\rightarrow$ Factor	type is copied up from the child	int
5	Expression $\rightarrow$ Term + Term	mixing float and int promotes the result to float	float
6	Assignment $\rightarrow$ x = Expression	storing a float into an int variable is allowed by our rule	Passed

6. Program Files

File	Purpose
task1_lexeme_separation.py	Reads the source and splits it into lexemes (Task 1)
task2_lexeme_grouping.py	Sorts lexemes into keyword, function, variable, operator and other groups (Task 2)
task3_lookup_table.py	Assigns id numbers to variables and prints the lookup table (Task 3)
task4_parser_semantic.py	Parser, parse tree, and attribute-grammar type checking (Task 4)

7. Challenges Encountered and Fixes Applied

Challenge	Fix applied
Two-character operators like $\leq$ and $++$ were splitting into two separate lexemes.	The two-character operator list is checked before single characters — the longest-match rule.
A function name and a variable name look identical to the scanner.	Peek one lexeme ahead: if the next lexeme is '(', the name is a function; otherwise it's a variable.
The same variable kept showing up repeatedly in the group listing and the lookup table.	set() removes duplicates, and the result is sorted for a clean listing.

Challenge	Fix applied
The Expression grammar rule is left-recursive, sending the parser into infinite recursion.	Left recursion was eliminated by reading one term first, then looping over the remaining operators.
In <code>int a, b=5;</code> , only <code>a</code> was picking up the <code>int</code> type.	<code>current_type</code> is kept in a variable and only cleared at <code>;</code> or <code>)</code> , so <code>b</code> inherits <code>int</code> as well.
The parser used to halt at the first error, hiding every problem after it.	All errors are collected in a list, and parsing continues to the end so every issue surfaces in one run.

8. Conclusion

This assignment implemented the four core building blocks of a compiler front-end, and each one was validated against the sample inputs supplied with the brief. The lexeme separator recovered all 58 lexemes in their original order; the grouping stage reproduced the same five categories as the expected output; the identifier-numbering stage generated correct ids along with a complete lookup table; and the parser both built a proper parse tree and caught both type errors in the given test cases while correctly passing the well-typed program.

The biggest takeaway from this exercise is how tightly these phases depend on one another. The scanner alone cannot decide whether a name is a function or a variable; the parser alone cannot decide whether a program is meaningful; and the semantic checker cannot function at all without the symbol table built earlier in the pipeline. It's only when all of these pieces cooperate that a compiler actually understands the program it's reading. Building each phase by hand, rather than relying on a parser-generator tool, made it much clearer why compilers are structured into separate phases to begin with.